Word Document Manipulation with python-docx

Critical: Split Placeholder Problem

The #1 issue with Word templates: Word often splits placeholder text across multiple XML runs. For example, `{{CANDIDATE_NAME}}` might be stored as:
- Run 1: `{{CANDI`
- Run 2: `DATE_NAME}}`

This happens due to spell-check, formatting changes, or Word's internal XML structure.

Naive Approach (FAILS on split placeholders)

```python
# DON'T DO THIS - won't find split placeholders
for para in doc.paragraphs:
for run in para.runs:
if '{{NAME}}' in run.text: # Won't match if split!
run.text = run.text.replace('{{NAME}}', value)
```

Correct Approach: Paragraph-Level Search and Rebuild

```python
import re

def replace_placeholder_robust(paragraph, placeholder, value):
"""Replace placeholder that may be split across runs."""
full_text = paragraph.text
if placeholder not in full_text:
return False

# Find all runs and their positions
runs = paragraph.runs
if not runs:
```

```
return False

# Build mapping of character positions to runs
char_to_run = []
for run in runs:
for char in run.text:
char_to_run.append(run)

# Find placeholder position
start_idx = full_text.find(placeholder)
end_idx = start_idx + len(placeholder)

# Get runs that contain the placeholder
if start_idx >= len(char_to_run):
return False

start_run = char_to_run[start_idx]

# Clear all runs and rebuild with replacement
new_text = full_text.replace(placeholder, str(value))

# Preserve first run's formatting, clear others
for i, run in enumerate(runs):
if i == 0:
run.text = new_text
else:
run.text = ''

return True
```

Best Practice: Regex-Based Full Replacement

```python
import re
from docx import Document

def replace_all_placeholders(doc, data):
"""Replace all {{KEY}} placeholders with values from data dict."""

def replace_in_paragraph(para):
"""Replace placeholders in a single paragraph."""
text = para.text
# Find all placeholders
pattern = r'\{\{([A-Z_]+)\}\}'
matches = re.findall(pattern, text)

if not matches:
return

# Build new text with replacements
new_text = text
for key in matches:
placeholder = '{{' + key + '}}'
if key in data:
new_text = new_text.replace(placeholder, str(data[key]))

# If text changed, rebuild paragraph
```

```
if new_text != text:
# Clear all runs, put new text in first run
runs = para.runs
if runs:
runs[0].text = new_text
for run in runs[1:]:
run.text = ''

# Process all paragraphs
for para in doc.paragraphs:
replace_in_paragraph(para)

# Process tables (including nested)
for table in doc.tables:
for row in table.rows:
for cell in row.cells:
for para in cell.paragraphs:
replace_in_paragraph(para)
# Handle nested tables
for nested_table in cell.tables:
for nested_row in nested_table.rows:
for nested_cell in nested_row.cells:
for para in nested_cell.paragraphs:
replace_in_paragraph(para)

# Process headers and footers
for section in doc.sections:
for para in section.header.paragraphs:
replace_in_paragraph(para)
for para in section.footer.paragraphs:
replace_in_paragraph(para)
```

Headers and Footers

Headers/footers are separate from main document body:

```python
from docx import Document

doc = Document('template.docx')

# Access headers/footers through sections
for section in doc.sections:
# Header
header = section.header
for para in header.paragraphs:
# Process paragraphs
pass

# Footer
footer = section.footer
for para in footer.paragraphs:
# Process paragraphs
pass
```

Nested Tables

Tables can contain other tables. Must recurse:

```python
def process_table(table, data):
"""Process table including nested tables."""
for row in table.rows:
for cell in row.cells:
# Process paragraphs in cell
for para in cell.paragraphs:
replace_in_paragraph(para, data)

# Recurse into nested tables
for nested_table in cell.tables:
process_table(nested_table, data)
```

Conditional Sections

For `{{IF_CONDITION}}...{{END_IF_CONDITION}}` patterns:

```python
def handle_conditional(doc, condition_key, should_include, data):
"""Remove or keep conditional sections."""
start_marker = '{{IF_' + condition_key + '}}'
end_marker = '{{END_IF_' + condition_key + '}}'

for para in doc.paragraphs:
text = para.text
if start_marker in text and end_marker in text:
if should_include:
# Remove just the markers
new_text = text.replace(start_marker, '').replace(end_marker, '')
# Also replace any placeholders inside
for key, val in data.items():
new_text = new_text.replace('{{' + key + '}}', str(val))
else:
# Remove entire content between markers
new_text = ''

# Apply to first run
if para.runs:
para.runs[0].text = new_text
for run in para.runs[1:]:
run.text = ''
```

Complete Solution Pattern

```python
from docx import Document
import json
import re

def fill_template(template_path, data_path, output_path):
"""Fill Word template handling all edge cases."""

# Load data
```

```
with open(data_path) as f:
data = json.load(f)

# Load template
doc = Document(template_path)

def replace_in_para(para):
text = para.text
pattern = r'\{\{([A-Z_]+)\}\}'
if not re.search(pattern, text):
return

new_text = text
for match in re.finditer(pattern, text):
key = match.group(1)
placeholder = match.group(0)
if key in data:
new_text = new_text.replace(placeholder, str(data[key]))

if new_text != text and para.runs:
para.runs[0].text = new_text
for run in para.runs[1:]:
run.text = ''

# Main document
for para in doc.paragraphs:
replace_in_para(para)

# Tables (with nesting)
def process_table(table):
for row in table.rows:
for cell in row.cells:
for para in cell.paragraphs:
replace_in_para(para)
for nested in cell.tables:
process_table(nested)

for table in doc.tables:
process_table(table)

# Headers/Footers
for section in doc.sections:
for para in section.header.paragraphs:
replace_in_para(para)
for para in section.footer.paragraphs:
replace_in_para(para)

doc.save(output_path)

# Usage
fill_template('template.docx', 'data.json', 'output.docx')
```

Common Pitfalls

1. **Forgetting headers/footers** - They're not in `doc.paragraphs`
2. **Missing nested tables** - Must recurse into `cell.tables`
3. **Split placeholders** - Always work at paragraph level, not run level

4. **Losing formatting** - Keep first run's formatting when rebuilding
5. **Conditional markers left behind** - Remove `{{IF_...}}` markers after processing